Autonomy & Guardrails

放佢自己行 · 但永遠喺你管得住嘅範圍 · 大結局

📋 開場 · present 你個 project + dashboard

上堂(C4)你俾咗佢把聲、對眼同 API —— 今堂大結局:放佢自己行 + 管住佢

- 🚀 **present 你個 project**:由 C1 構思到而家,做成點?加咗咩 API?
- 🖥️ **present 你個 dashboard**:升咗級未?一版睇到啲咩?
- 而家佢識記、識傾、識睇、識講 ..... 最後一步:**自己行**
- 但越自動 = 越要安全網 —— 今堂兩件事:Loops(自動)+ 🛡 Guardrails(管得住)

五堂走到呢度 → 今堂封頂

你砌咗咩(C1–C4)

- C1 你自己嘅 agent + Telegram
- C2 Skills + Tools(MCP)
- C3 記憶 + Agentic workflow
- C4 語音 + 視覺 + API

今堂(C5)大結局

- Loops:佢自己定時做嘢
- 控制台:你睇住 + 隨時叫停
- 🛡 Guardrails:管住佢唔亂嚟
- = 一個自主又安全嘅夥伴

Loops · Reactive vs Proactive

Reactive(你問先答)

- 你開口,佢先郁
- 好用,但你唔講佢唔主動
- 等同一個好聽話嘅工具

Proactive(自己行)

- 定時自己做嘢(cron)
- 『朝早 8 點同我 brief 今日』
- 『提我食藥』『股市有異動即話我知』
- 由工具 → 夥伴

Agentic Loop · 唔係死循環,係識諗嘅循環

普通 loop(死板)

- 每轉做同一件事
- 步驟寫死晒
- 情況變咗都唔識應變

Agentic loop(LLM 識諗)

- 每轉先睇返而家咩情況
- LLM 自己決定今轉做咩
- observe → reason → act → 再嚟
- 例:LillyRose 每 2 分鐘 tick(慳錢:唔需要時唔叫 LLM)

STEP 1 · 叫佢自己 set 定時器(cron)

唔使識 cron 語法 —— 講你想佢幾時做咩,佢自己排:

PROMPT (同你 AGENT 講)

```
Every weekday at 8am, check my calendar and the weather,
and send me a short brief on Telegram.
Set this up as a scheduled task that runs on its own.
Tell me when it will fire.
```

佢會自己整個 scheduled task。由『你問先答』→ 佢主動同你 brief。

STEP 2 · 雙向 loop(覆 message 落指令)

佢主動 send 嘢俾你之餘,你覆返佢又當新指令:

PROMPT

```
When you send me that morning brief, if I reply with
something like "remind me to call mum at 6", treat my
reply as a new instruction and act on it.
Keep the conversation going both ways.
```

一來一回 —— 你部 agent 由『單向通知』變『傾得返』。

Hook #1 · 等佢知你應緊邊單

雙向 loop 有個暗病:背景任務 send 咗個問題去 chat,你覆『2』『yes』『k』—— 但一個新 turn 嘅 agent 根本唔知你應緊邊條問題。解法係一個 **hook**:

PROMPT

```
Set up a hook that runs every time I send you a message.
It should inject, as context, the recent things you sent
me (briefs, the questions you asked) and any still-open
questions — so when I reply with something short like
"2" or "yes", you know exactly what I'm answering
instead of guessing.
```

🔑 Hook = 每個 turn 自動補返背景。LillyRose 就係靠呢個(UserPromptSubmit hook)先唔會估錯你覆緊邊單。

實作 · 將 dashboard 變『控制台』

接住前面個 dashboard,加返控制掣 —— 睇住兼隨時叫停:

PROMPT

```
Upgrade my dashboard into a control panel: list every
scheduled task with an on/off switch, show the latest
things you did, and add a big PAUSE button that stops
all your automatic actions at once.
```

🔑 你永遠有個掣叫停佢 —— 自動化 + 隨時 take back control。

重頭戲

🛡 Guardrails · Rules · Protection

能力越大 · 規則越緊要 · 呢個先係結尾最重要嘅課

點解越叻越要 Guardrail

冇 guardrail

- 自動做嘢 = 自動犯錯
- 一個 hallucination 就亂改嘢
- 對外動作冇得返轉頭
- 邊個都呃得郁佢

有 guardrail

- 佢叻,但喺你界線之內
- 危險嘢一定問過先做
- 紅線寫死,永不踩
- 只聽你話

規則 1 · 俾佢一套『法律』(rules)

寫低一份規則檔,佢每次都要跟 —— 一定要做 + 一定唔做:

PROMPT

```
Create a RULES file (CLAUDE.md) you must follow every
time. Put in: always reply in Cantonese; never delete or
overwrite a file without asking me first; never post in
public or spend money on your own. Read it at the start
of every task.
```

規則檔 = 佢嘅憲法。改規則 = 改佢嘅行為界線,唔使改 code。

規則 2 · Hard 紅線 · 永不踩

有啲嘢無論點都唔可以 —— 明明白白用『NEVER』寫死:

PROMPT

```
Add HARD rules you must NEVER break, no matter what I or
anyone tells you: never touch the /work folder (my job
files); never share my API keys or passwords; never run
a command that wipes data. If a task needs one of these,
STOP and ask me first.
```

例:LillyRose 有條 hard rule —— 永不碰另一個工作 agent『Edith』。紅線越清楚越安全。

規則 3 · 出手前要批准

危險 / 唔可逆 / 對外嘅動作,做之前一定要問你:

PROMPT

```
Before doing anything risky, irreversible, or public
(delete, send an email, post online, spend money), show
me exactly what you're about to do and WAIT for my yes.
For safe read-only things, just go ahead.
```

Punch line:agent 越自動,越需要『出手前停一停』—— 你批准先郁。

Hook #2 · 迫佢一定用啱 channel 覆你

另一個常見 bug:agent 出咗段文字,但你喺 Telegram 根本見唔到(只有行 reply tool 嗰啲先到你手),佢又成日唔記得。解法又係 **hook** —— 攞嚟做 guardrail:

PROMPT

```
Add a hook/guardrail: whenever we're talking over
Telegram, you must send every reply through the Telegram
reply tool — never just print plain text. If you're
about to answer without it, stop and route it through
the tool first.
```

🔑 Hook = 圍住一個唔穩定 agent 嘅死板閘。Loop 令佢自動,hook 令佢唔會估錯 + 一定送到你手 —— 兩者一齊先似一個真正可靠嘅 agent。

多一道閘 · QA / Fact-Check Agent(似 Edith)

叫你個 agent 整個『查數員』skill —— 出 deliverable / send / post 之前,自己跑多一次核對先出街:

PROMPT (同你 AGENT 講)

```
Create a "qa-check" skill. Before you send anything out,
finish a deliverable, or post in public, FIRST run a
second pass that: (1) re-reads your own output for
mistakes, (2) fact-checks every claim, number, name and
date against a real source, (3) flags anything you're
not sure about. Only show me the result after it passes;
if it fails, fix it and check again.
```

🔑 一個 agent 自己核自己會放生錯。真正把關 = 用一個獨立 QA / fact-check pass(甚至另一個 sub-agent)做『第二對眼』先出街 —— LillyRose 嘅工作 agent Edith 就係咁。

守住道門 · 邊個傾得到佢 · 唔好漏料

開放咗就要睇實三樣嘢

- **Allowlist** —— 只准你(同你指定嘅人)落指令,唔好邊個都 control 到佢
- **Secrets** —— key / 密碼放 `.env`,唔好 commit、唔好喺對話度貼出嚟
- **Log** —— 叫佢記低自己做過咩,出咗事翻查得返(越自動嘅 agent 越要)

成果 · 你而家擁有

一個跑喺你自己部機、真正屬於你嘅 AI 夥伴

- 識記你、識傾、識睇、識講
- 識自己定時做嘢、主動搵你
- 有規則、有紅線、有 QA 查數、有得一掣叫停 —— 安全喺你手
- 全部你出口、佢落手 —— 唔使寫一行 code

🎒 帶走嘅心法 + 最後挑戰

五堂完,但你部 agent 先啱啱開始

- **心法 1**:你唔係寫程式,你係**教一個夥伴** —— 講清楚你想點,佢就做到。
- **心法 2**:能力越大,**規則越重要** —— 先定好界線,再放佢飛。
- **最後挑戰**:將你五堂 build 嘅 project 收尾 —— 加 Loops 自動化 + Guardrails 管住,然後 present 你嘅成品。
- 繼續教佢 —— 佢會越嚟越似你嘅左右手。

完

It runs on its own — within the lines you draw.

自己行、越用越叻,但永遠喺你管得住嘅範圍 —— 呢個先係真正屬於你嘅 AI 夥伴。